ineering

PY103

Wrap-up I

Yesterday

Welcome to your next lesson: Wrap-up I.

I'm preparing your personalized lesson using what you've already learned, what's coming up, and the best ways to help everything stick.

Keep going, you’re doing great.

Ron, this is your final OOP mission for this unit 🎯

We’re going to build a small “object-powered system” that uses:

classes + objects
encapsulation (internal state + getters/setters)
inheritance + overriding (polymorphism)
composition (one object managing others)
__str__ for clean output
To keep it concrete, pick one scenario you like most:

A) Task management system (Tasks, UrgentTask, TaskList)

B) Course system (Course, OnlineCourse, InPersonCourse, Program)

C) Library system (Item, Book, Movie, Library)

D) You propose your own scenario with 2–3 related classes

Reply with just a letter (A/B/C/D).

If D, add a 1-sentence description of your idea 🙂

from ray trace graphics work, a 3d point , has x y z data methods to retrieve and alter them, a colored pixel woukd use it as a parent but add a coolor attrib and a print to screen, also a line woud be 2 3d points with a vector dictating its angle,methods to move it which i turn all the 3d point methods, a plane class built off 4 lines, now adds area (L * W) and alsomove rotate features that all line moves that fak 3d point moves

Oh this is a fantastic scenario for OOP, Ron 🔥

You basically described a tiny ray-trace engine using classes.

Let me restate your idea in clean steps and we'll shape it into the challenge:

Point3D class

stores: x, y, z (internally, like _x, _y, _z)
methods: getters, setters / move methods to change position
ColoredPixel class (inherits from Point3D)

adds: color attribute
overrides: __str__ (your "print to screen") to show position + color
Line3D class (uses composition)

has: 2 Point3D objects: start_point, end_point
also: maybe a simple direction vector or we just infer direction from the two points
methods: move line (by moving both points)
Plane class

has: 4 Line3D objects (composition again)
methods:
compute area (L × W) — we can simplify length/width for now

move plane (moves all lines → which move all points)
Before we touch code, quick relationship check:

ColoredPixel is-a Point3D (inheritance)
Line3D has-a Point3D (two of them)
Plane has-a Line3D (four of them)

Does that matching of is-a vs has-a feel right to you?

If yes, next step I'll sketch a small class diagram-style table and then we'll move to the code editor to implement it step by step 💻

Yes, that makes sense

Nice, then let's turn your idea into a clear little "map" before you code anything 🧭

Here's a simple layout:

Class	Internal data (state)	Relationship type	Main responsibilities
Point3D	_x, _y, _z	base class	Store 3D position, move/change coordinates
ColoredPixel	_x, _y, _z, _color	is-a Point3D	Same as Point3D + color + nice __str__
Line3D	_start_point, _end_point (both Point3D)	has-a Point3D	Represent a segment, move both endpoints
Plane	_lines (list of 4 Line3D objects)	has-a Line3D	Represent rectangle-like plane, area, move

We'll keep the math simple (no hardcore 3D rotations) and focus on OOP: encapsulation, inheritance, composition, polymorphism, __str__.

Next step: we start coding with just Point3D in the editor.

In the code editor, create a Point3D class that:

Stores _x, _y, _z in __init__
Has getter methods like get_x, get_y, get_z
Has a move(dx, dy, dz) method that adds to each coordinate

Implements __str__ to return something like: "Point3D(x=1, y=2, z=3)"

Write that in the editor, hit Run to check for errors, then Submit code when Point3D is done.

I'll review it and then we'll add ColoredPixel on top 💡

Python

```
class Point3d:
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z

    def get_x(self):
        return self.x

    def get_y(self):
        return self.y

    def get_z(self):
        return self.z

    def move(self, dx, dy, dz):
        self.x = self.x + dx
        self.y = self.y + dy
        self.z = self.z + dz

    def __str__(self):
        return f"Point3D(x={self.x},y={self.y}, z={self.z})"


point = Point3d(4, 5, 1)
print(point)
```

Nice work, Ron 🌟

Your Point3d:

Has a constructor
Getters
move method
Clean __str__
And it runs correctly 👍

Two small OOP polish points (not required, but more “encapsulated” style):

Use internal names: self._x, self._y, self._z
Then getters return those: return self._x, etc.

If you’re okay with that, you can quickly rename x→_x, y→_y, z→_z in your class. Want to do that tweak now, or keep going as-is and move on to ColoredPixel?

Python

```
class Point3d:
    def __init__(self, x, y, z):
        self._x = x
        self._y = y
        self._z = z

    def get_x(self):
        return self._x

    def get_y(self):
        return self._y

    def get_z(self):
        return self._z

    def move(self, dx, dy, dz):
        self._x = self._x + dx
        self._y = self._y + dy
        self._z = self._z + dz

    def __str__(self):
        return f"Point3D(x={self._x},y={self._y},
z={self._z})"


point = Point3d(4, 5, 1)
print(point)
```

Beautiful update, Ron — this is a clean, encapsulated Point3d now ✅

You’ve got:

internal state: _x, _y, _z

```
getters
move that changes state
readable __str__
Next step: add ColoredPixel that inherits from Point3d 🎨


In the editor, under your Point3d class, add:


a ColoredPixel class that:
takes x, y, z, color in __init__
calls the Point3d constructor “manually” (no super()):
store _x, _y, _z just like Point3d
stores _color
overrides __str__ to return something like:
"ColoredPixel(x=4, y=5, z=1, color='red')"

Then, at the bottom, instead of (or in addition to) your
point, create and print one ColoredPixel to check it.


Go ahead and implement that in the code editor, run, then
Submit code so I can look at it 👀

Today

Python
class Point3d:
    """ Store and manipulate a 3d X,Y,Z coordinate
    methods: get_x, get_y, get_z return x,y,or z
            move(newx,newy,newz) translates x+newx,y+newy,
z+newz
    """

    #intitialie and store x,y, z
    def __init__(self, x, y, z):
        self._x = x
        self._y = y
        self._z = z
    #return x
    def get_x(self):
        return self._x
    #return y
    def get_y(self):
        return self._y
    #return z
    def get_z(self):
```

```
            return self._z
        #trnslate x,y,z to new x,y,z
        def move(self, dx, dy, dz):
            self._x = self._x + dx
            self._y = self._y + dy
            self._z = self._z + dz

        # Return state of point object
        def __str__(self):
            return f"Point3D(x={self._x},y={self._y},
    z={self._z})"


    class ColoredPixel(Point3d):
        """ Store and manipulate a 3d colored pixel
        adds color to a Point3d object it contains
        methods:
        set_color and get_color as well as al Point3d methods
        """
        #Initialize, store x,y,z in Point3d and color locally
        def __init__(self, x, y, z, color):
            self._point = Point3d(x, y, x)
            self._color = color

        #Change color ofpixel
        def set_color(self, color):
            self._color = color

        #Report current color
        def get_color(self):
            return self._color

        #Return state of ColoredPixel object
        def __str__(self):
            return f"ColoredPixel(x={point.get_x()},
    y={point.get_y()}, z={point.get_z()}, color='{self._color}'"


    point = Point3d(4, 5, 1)
    red_pixel = ColoredPixel(1, 1, 0, "red")
    print(point)
    print(red_pixel)


    Ron, this is very close and you’re doing good stuff here 💪

    Let’s fix two key things step by step.
```

First: the design of ColoredPixel.

Right now you wrote:

Python
```
class ColoredPixel(Point3d):
    ...
    def __init__(self, x, y, z, color):
        self._point = Point3d(x, y, x)
        self._color = color
```
But you said yourself: “a colored pixel would use it as a parent”.

That means we want to reuse Point3d’s _x, _y, _z, not hide a separate _point.

So for this exercise, let’s keep it simple:

Let ColoredPixel truly be a Point3d (inheritance)
Don’t store another Point3d inside it
Just store _color in addition

Also, in __str__ you used point.get_x() etc., but point there is the global variable at the bottom, not the colored pixel itself. We want to use self.

Here’s the minimal change I want you to make in the editor:

In ColoredPixel.__init__, instead of creating self._point = Point3d(...), directly set:

```
self._x = x
self._y = y
self._z = z
self._color = color
```
In __str__, use self.get_x() / self.get_y() / self.get_z() and self._color.

So rewrite ColoredPixel to follow that idea (don’t copy from chat, retype it in the editor), run, then Submit code again.

After that, we’ll add Line3D on top of these 🚀

Python

```
class Point3d:
    """Store and manipulate a 3d X,Y,Z coordinate
    methods: get_x, get_y, get_z return x,y,or z
            move(newx,newy,newz) translates x+newx,y+newy,
z+newz
    """

    # intitialie and store x,y, z
    def __init__(self, x, y, z):
        self._x = x
        self._y = y
        self._z = z

    # return x
    def get_x(self):
        return self._x

    # return y
    def get_y(self):
        return self._y

    # return z
    def get_z(self):
        return self._z

    # trnslate x,y,z to new x,y,z
    def move(self, dx, dy, dz):
        self._x = self._x + dx
        self._y = self._y + dy
        self._z = self._z + dz

    # Return state of point object
    def __str__(self):
        return f"Point3D(x={self._x},y={self._y},
z={self._z})"


class ColoredPixel(Point3d):
    """Store and manipulate a 3d colored pixel
    adds color to a Point3d object it contains
    methods:
    set_color and get_color as well as all Point3d methods
    """

```

```
    # Initialize, store x,y,z in Point3d and color locally
    def __init__(self, x, y, z, color):
        self._x = x
        self._y = y
        self._z = z
        self._color = color

    # Change color ofpixel
    def set_color(self, color):
        self._color = color

    # Report current color
    def get_color(self):
        return self._color

    # Return state of ColoredPixel object
    def __str__(self):
        return (
            f"ColoredPixel(x={self._x}, y={self._y},
z={self._z}, color='{self._color}'"
        )


point = Point3d(4, 5, 1)
red_pixel = ColoredPixel(1, 1, 0, "red")
print(point)
print(red_pixel)
This is coming together really well, Ron 🔧🎨


Quick notes on what you have now:


Point3d looks solid: internal _x, _y, _z, getters, move, and
__str__ all good.
ColoredPixel:
Correctly stores _x, _y, _z, _color
Has set_color / get_color
Nicely overrides __str__ (tiny missing ) at the very end, but
your run still showed the string, so good catch by the
interpreter).
Next step: let’s add Line3D, which will use composition:


It has two Point3d objects: _start_point, _end_point
Methods:
get_start() and get_end() (return those points)
```

```
move(dx, dy, dz) → calls move on both points
__str__ → something like
"Line3D(start=Point3D(...), end=Point3D(...))"

In the editor, under ColoredPixel, add a Line3d class with
those ideas, then at the bottom:


Create two Point3ds
Build one Line3d from them
Print the line
Call line.move(...)
Print the line again

Then run and Submit code so I can see it.

Python
class Point3d:
    """Store and manipulate a 3d X,Y,Z coordinate
    methods: get_x, get_y, get_z return x,y,or z
            move(newx,newy,newz) translates x+newx,y+newy,
z+newz
    """

    # intitialie and store x,y, z
    def __init__(self, x, y, z):
        self._x = x
        self._y = y
        self._z = z

    # return x
    def get_x(self):
        return self._x

    # return y
    def get_y(self):
        return self._y

    # return z
    def get_z(self):
        return self._z

    # trnslate x,y,z to new x,y,z
    def move(self, dx, dy, dz):
        self._x = self._x + dx
        self._y = self._y + dy
        self._z = self._z + dz
```

```
467
468         # Return state of point object
469         def __str__(self):
470             return f"Point3D(x={self._x},y={self._y},
    z={self._z})"
471
472
473     class ColoredPixel(Point3d):
474         """Store and manipulate a 3d colored pixel
475         adds color to a Point3d object it contains
476         methods:
477         set_color and get_color as well as al Point3d methods
478         """
479
480         # Initialize, store x,y,z in Point3d and color locally
481         def __init__(self, x, y, z, color):
482             self._x = x
483             self._y = y
484             self._z = z
485             self._color = color
486
487         # Change color ofpixel
488         def set_color(self, color):
489             self._color = color
490
491         # Report current color
492         def get_color(self):
493             return self._color
494
495         # Return state of ColoredPixel object
496         def __str__(self):
497             return (
498                 f"ColoredPixel(x={self._x}, y={self._y},
    z={self._z}, color={self._color})"
499             )
500
501
502     class Line3d:
503         """Store and manipulate a 3d line segment
504         methods: get_start, get_end return start or end Point3d
505                  move(newx,newy,newz) translates start and end
    points
506         """
507
508         # Initialize and store start and end Point3d objects
509         def __init__(self, start, end):
510             self._start_point = Point3d(start.get_x(),
```

```
    start.get_y(), start.get_z())
511         self._end_point = Point3d(end.get_x(), end.get_y(),
    end.get_z())
512
513     # Return start point
514     def get_start(self):
515         return self._start_point
516
517     # Return end point
518     def get_end(self):
519         return self._end_point
520
521     # Translate both start and end points by dx,dy,dz
522     def move(self, dx, dy, dz):
523         self._start_point.move(dx, dy, dz)
524         self._end_point.move(dx, dy, dz)
525
526     # Return state of Line3d object
527     def __str__(self):
528         return
    f"Line3D(start=Point3d(x={self._start_point.get_x()},
    y={self._start_point.get_y()},
    z={self._start_point.get_z()}),
    end=Point3d(x={self._end_point.get_x()},
    y={self._end_point.get_y()}, z={self._end_point.get_z()}))"
529
530 point1 = Point3d(1, 2, 3)
531 point2 = Point3d(4, 5, 1)
532
533 red_pixel = ColoredPixel(1, 1, 0, "red")
534 line = Line3d(point1, point2)
535 print(point1)
536 print(red_pixel)
537 print(line)
538
539 This is strong work, Ron 🔥
540
541
542 You’ve already got:
543
544
545 Encapsulation: _x, _y, _z inside Point3d
546 Inheritance: ColoredPixel(Point3d)
547 Composition: Line3d has two Point3ds
548 Readable __str__ for all three
549
550 Next and final big piece for this mission is the Plane 💪
```

Let’s keep it simple and rectangular:

Plane has 4 Line3d objects: _top, _bottom, _left, _right
It also knows a length and width so it can compute area = length * width
Methods idea:
get_area() → returns the area
move(dx, dy, dz) → calls move on all 4 lines
__str__ → something short like "Plane(area=..., lines=[...])" (you don’t need to print every coordinate if that’s too long)
Step-by-step task:

In the editor, under Line3d, create a Plane class with:

__init__(self, top, bottom, left, right, length, width)
store the 4 lines as internal attributes
store length and width
get_area(self) that returns length * width
move(self, dx, dy, dz) that calls move on all 4 lines
__str__ that prints the area and maybe a short description of the lines
At the bottom:

Reuse or create 4 Line3d objects to form a rectangle (they don’t need to be perfectly geometric; we’re focusing on OOP).
Create one Plane
Print the plane
Call plane.move(...)
Print the plane again

Then run and Submit code.

After that, we’ll do a quick state-trace and then wrap this lesson as a win 🏁

Python
```
class Point3d:
    """Store and manipulate a 3d X,Y,Z coordinate
    methods: get_x, get_y, get_z return x,y,or z
            move(newx,newy,newz) translates x+newx,y+newy, z+newz
    """
```

```
592       # intitialie and store x,y, z
593       def __init__(self, x, y, z):
594           self._x = x
595           self._y = y
596           self._z = z
597
598       # return x
599       def get_x(self):
600           return self._x
601
602       # return y
603       def get_y(self):
604           return self._y
605
606       # return z
607       def get_z(self):
608           return self._z
609
610       # trnslate x,y,z to new x,y,z
611       def move(self, dx, dy, dz):
612           self._x = self._x + dx
613           self._y = self._y + dy
614           self._z = self._z + dz
615
616       # Return state of point object
617       def __str__(self):
618           return f"Point3D(x={self._x},y={self._y},
      z={self._z})\n"
619
620
621   class ColoredPixel(Point3d):
622       """Store and manipulate a 3d colored pixel
623       adds color to a Point3d object it contains
624       methods:
625       set_color and get_color as well as al Point3d methods
626       """
627
628       # Initialize, store x,y,z in Point3d and color locally
629       def __init__(self, x, y, z, color):
630           self._x = x
631           self._y = y
632           self._z = z
633           self._color = color
634
635       # Change color ofpixel
636       def set_color(self, color):
637           self._color = color
```

```
638
639         # Report current color
640         def get_color(self):
641             return self._color
642
643         # Return state of ColoredPixel object
644         def __str__(self):
645             return f"ColoredPixel(x={self._x}, y={self._y},
    z={self._z}, color={self._color})\n"
646
647
648     class Line3d:
649         """Store and manipulate a 3d line segment
650         methods: get_start, get_end return start or end Point3d
651                 move(newx,newy,newz) translates start and end
    points
652         """
653
654         # Initialize and store start and end Point3d objects
655         def __init__(self, start, end):
656             self._start_point = Point3d(start.get_x(),
    start.get_y(), start.get_z())
657             self._end_point = Point3d(end.get_x(), end.get_y(),
    end.get_z())
658
659         # Return start point
660         def get_start(self):
661             return self._start_point
662
663         # Return end point
664         def get_end(self):
665             return self._end_point
666
667         # Translate both start and end points by dx,dy,dz
668         def move(self, dx, dy, dz):
669             self._start_point.move(dx, dy, dz)
670             self._end_point.move(dx, dy, dz)
671
672         # Return state of Line3d object
673         def __str__(self):
674             return
    f"Line3D(start=Point3d(x={self._start_point.get_x()},
    y={self._start_point.get_y()},
    z={self._start_point.get_z()}),
    end=Point3d(x={self._end_point.get_x()},
    y={self._end_point.get_y()}, z={self._end_point.get_z()}))\n"
675
```

```
676
677 class Plane3d:
678     """Store and manipulate a 3d plane
679     methods: get_area returns area of plane
680             move(newx,newy,newz) translates all four lines
681     """
682
683     # Initialize and store four Line3d objects and length and
    width of plane as local variables
684     def __init__(self, line1, line2, line3, line4, length,
    width):
685         self._line1 = Line3d(line1.get_start(),
    line1.get_end())
686         self._line2 = Line3d(line2.get_start(),
    line2.get_end())
687         self._line3 = Line3d(line3.get_start(),
    line3.get_end())
688         self._line4 = Line3d(line4.get_start(),
    line4.get_end())
689         self._length = length
690         self._width = width
691
692     # Calculate and return the area of the plane
693     def get_area(self):
694         return self._length * self._width
695
696     # Translate all four lines by dx,dy,dz
697     def move(self, dx, dy, dz):
698         self._line1.move(dx, dy, dz)
699         self._line2.move(dx, dy, dz)
700         self._line3.move(dx, dy, dz)
701         self._line4.move(dx, dy, dz)
702
703     # Return state of Plane3d object
704     def __str__(self):
705         return f"Plane3d:\n\tArea:
    {self.get_area()}\n\tLine1: {self._line1}\tLine2:
    {self._line2}\tLine3: {self._line3}\tLine4: {self._line4}"
706
707
708 point1 = Point3d(1, 1, 1)
709 point2 = Point3d(1, 6, 1)
710 point3 = Point3d(1, 6, 1)
711 point4 = Point3d(6, 6, 1)
712 point5 = Point3d(6, 6, 1)
713 point6 = Point3d(6, 1, 1)
714 point7 = Point3d(6, 1, 1)
```

```
point8 = Point3d(1, 1, 1)

red_pixel = ColoredPixel(1, 1, 0, "red")
line1 = Line3d(point1, point2)
line2 = Line3d(point3, point4)
line3 = Line3d(point5, point6)
line4 = Line3d(point7, point8)
plane = Plane3d(line1, line2, line3, line4, 5, 5)
print(point1)
print(red_pixel)
print(line1)
print(plane)
plane.move(1, 1, 1)
print(plane)

Ron, this is excellent work 🎯


You built a clean little 3D geometry “engine” using OOP:


Encapsulation:
Point3d hides _x, _y, _z and changes them only through move.
Inheritance:
ColoredPixel reuses the position idea and adds _color plus
its own __str__.
Composition:
Line3d is built from two Point3ds.
Plane3d is built from four Line3ds.
Polymorphism-style behavior:
The same method name move is used on Point3d, Line3d, and
Plane3d, and each one moves “its own” internal pieces in the
right way.
Readable output:
Your __str__ methods make the final printout easy to
understand.

You didn’t just follow a scaffold; you made design choices
and wired several classes together into a small system.
That’s exactly what this wrap-up was meant to prove 💪



If you want a quick final reflection (optional), answer this
in one line:


```

In your own words: what does Plane3d.move(dx, dy, dz) actually end up moving under the hood?

Otherwise, you're ready to hit Next Lesson when you're ready to move on 🚀

How would you rate this lesson?

it moves the x,y,z of each of the 4 lines it stores, by calling lines move, which updates start and end x,y,z of each line by calling point3ds move which moves each x,y,z by the called for translation distance

Perfect explanation, Ron ✅

You just traced the whole chain correctly:

Plane3d.move → calls move on each of its 4 Line3d objects
each Line3d.move → calls move on its 2 Point3d endpoints
each Point3d.move → actually changes the _x, _y, _z values

So one call at the top moves every single underlying point.

That's exactly the kind of layered, object-based thinking this unit was aiming for 🎯

You've effectively built:

a base 3D point
a specialized colored point
lines made of points
a plane made of lines
all with clean output and consistent movement.

You've mastered what this wrap-up wanted.

When you're ready, go ahead and move on to the next lesson 🚀


```
class Point3d:
    """Store and manipulate a 3d X,Y,Z coordinate
    methods: get_x, get_y, get_z return x,y,or z
             move(newx,newy,newz) translates x+newx,y+newy,
z+newz
```

```
795         """
796
797         # intitialie and store x,y, z
798         def __init__(self, x, y, z):
799             self._x = x
800             self._y = y
801             self._z = z
802
803         # return x
804         def get_x(self):
805             return self._x
806
807         # return y
808         def get_y(self):
809             return self._y
810
811         # return z
812         def get_z(self):
813             return self._z
814
815         # trnslate x,y,z to new x,y,z
816         def move(self, dx, dy, dz):
817             self._x = self._x + dx
818             self._y = self._y + dy
819             self._z = self._z + dz
820
821         # Return state of point object
822         def __str__(self):
823             return f"Point3D(x={self._x},y={self._y},
    z={self._z})\n"
824
825
826     class ColoredPixel(Point3d):
827         """Store and manipulate a 3d colored pixel
828         adds color to a Point3d object it contains
829         methods:
830         set_color and get_color as well as al Point3d methods
831         """
832
833         # Initialize, store x,y,z in Point3d and color locally
834         def __init__(self, x, y, z, color):
835             self._x = x
836             self._y = y
837             self._z = z
838             self._color = color
839
840         # Change color ofpixel
```

```
841       def set_color(self, color):
842           self._color = color
843
844       # Report current color
845       def get_color(self):
846           return self._color
847
848       # Return state of ColoredPixel object
849       def __str__(self):
850           return f"ColoredPixel(x={self._x}, y={self._y},
      z={self._z}, color={self._color})\n"
851
852
853   class Line3d:
854       """Store and manipulate a 3d line segment
855       methods: get_start, get_end return start or end Point3d
856                move(newx,newy,newz) translates start and end
      points
857       """
858
859       # Initialize and store start and end Point3d objects
860       def __init__(self, start, end):
861           self._start_point = Point3d(start.get_x(),
      start.get_y(), start.get_z())
862           self._end_point = Point3d(end.get_x(), end.get_y(),
      end.get_z())
863
864       # Return start point
865       def get_start(self):
866           return self._start_point
867
868       # Return end point
869       def get_end(self):
870           return self._end_point
871
872       # Translate both start and end points by dx,dy,dz
873       def move(self, dx, dy, dz):
874           self._start_point.move(dx, dy, dz)
875           self._end_point.move(dx, dy, dz)
876
877       # Return state of Line3d object
878       def __str__(self):
879           return
      f"Line3D(start=Point3d(x={self._start_point.get_x()},
      y={self._start_point.get_y()},
      z={self._start_point.get_z()}),
      end=Point3d(x={self._end_point.get_x()},
```

```
    y={self._end_point.get_y()}, z={self._end_point.get_z()}))\n"
880
881
882 class Plane3d:
883     """Store and manipulate a 3d plane
884     methods: get_area returns area of plane
885             move(newx,newy,newz) translates all four lines
886     """
887
888     # Initialize and store four Line3d objects and length and
    width of plane as local variables
889     def __init__(self, line1, line2, line3, line4, length,
    width):
890         self._line1 = Line3d(line1.get_start(),
    line1.get_end())
891         self._line2 = Line3d(line2.get_start(),
    line2.get_end())
892         self._line3 = Line3d(line3.get_start(),
    line3.get_end())
893         self._line4 = Line3d(line4.get_start(),
    line4.get_end())
894         self._length = length
895         self._width = width
896
897     # Calculate and return the area of the plane
898     def get_area(self):
899         return self._length * self._width
900
901     # Translate all four lines by dx,dy,dz
902     def move(self, dx, dy, dz):
903         self._line1.move(dx, dy, dz)
904         self._line2.move(dx, dy, dz)
905         self._line3.move(dx, dy, dz)
906         self._line4.move(dx, dy, dz)
907
908     # Return state of Plane3d object
909     def __str__(self):
910         return f"Plane3d:\n\tArea:
    {self.get_area()}\n\tLine1: {self._line1}\tLine2:
    {self._line2}\tLine3: {self._line3}\tLine4: {self._line4}"
911
912
913 point1 = Point3d(1, 1, 1)
914 point2 = Point3d(1, 6, 1)
915 point3 = Point3d(1, 6, 1)
916 point4 = Point3d(6, 6, 1)
917 point5 = Point3d(6, 6, 1)
```

```
  point6 = Point3d(6, 1, 1)
  point7 = Point3d(6, 1, 1)
  point8 = Point3d(1, 1, 1)

  red_pixel = ColoredPixel(1, 1, 0, "red")
  line1 = Line3d(point1, point2)
  line2 = Line3d(point3, point4)
  line3 = Line3d(point5, point6)
  line4 = Line3d(point7, point8)
  plane = Plane3d(line1, line2, line3, line4, 5, 5)
  print(point1)
  print(red_pixel)
  print(line1)
  print(plane)
  plane.move(1, 1, 1)
  print(plane)


```